

Muhammad Hasnain Naeem

Pr Report

 Quick Submit Quick Submit National University of Computer and Emerging Sciences, Islamabad

Document Details

Submission ID

trn:oid::1:3424266385

Submission Date

Nov 25, 2025, 2:06 PM GMT+5

Download Date

Nov 25, 2025, 2:11 PM GMT+5

File Name

ProjectReportPFLab_Edit.docx

File Size

21.8 KB

6 Pages**926 Words****5,227 Characters**

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



0 AI-generated only 0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Final Project Report — Programming Fundamentals Lab

University Name: National University of Computer and Emerging Sciences (FAST-NUCES)

Department: Department of Software Engineering (BSSE)

Course: Programming Fundamentals -Lab

Project Title: All In One Calculator

Submitted By: Muhammad Hasnain (Roll No. 25K-3029), Sohail Ahmed (Roll No. 25K-3015)

Submitted To: Ms. Ravia Ejaz

Semester: Fall 2025

Date: 23/11/2025

Abstract

We have made a console based All In One Calculator in C programming language with various modes which include Scientific Mode, Unit Conversion Mode, Graphing Utility, and the starring feature, Ai Mode which responds to your queries. We used different types of algorithms to implement operator precedence based on Mathematics rules (e.g BODMAS). We used windows.h header file to run command prompt commands through code and implement an appealing console UI. We also used curl header file which is a windows built in program to send and fetch http requests through which we send requests to the Google provided Gemini API along with the user's queries (the user can also upload images) and send back response which is then converted from JSON to simple HTML and then opened in the browser as an HTML file. This project made us learn practical use of different algorithms and coding techniques and different C libraries. We also learned the use of APIs and interacting with them through C programming language.

1. Introduction

When we talked to different batch mates and looked around us, we found that the students were having trouble in studying Calculus because of the many disconnected apps they use like separate apps for simple calculator, scientific calculator, graphing calculator, Ai solver, etc. So we decided to change this and help all our batch mates and we made this All in One Calculator which allows you to use any type of calculator from scientific to Ai solver even the conversion calculator and graphing calculator. Now the user wont have to switch between different apps, making both studying and user experience great!

2. Objectives

- Provide better user experience.
- Bring all of the different utilities to a single platform.
- Help all of the students.

3. System Design

System Overview

The program runs in a loop, providing 4 different modes; 1. Scientific, 2. Conversion, 3. Graphing, 4. Ai Agent. The user can use any mode as many times as he wants and then get back to main menu to switch modes through a very easy command.

How the program flows:

Start → Main Menu → Pick a Mode (Scientific / Converter / Graphing / Ai Agent) → Type Problems → See the Output → Go Back to Menu or Exit.

Algorithm (Scientific Mode)

1. Input: You type in something like $10 + 2 * 5$. (We use safe input method by clearing the input buffer first.)
2. Conversion: The Shunting-yard algorithm converts from human notation (traditional maths notation) to Postfix notation (notation which is easy to understand for the computer).
4. Evaluation: The converted expression is then calculated and result is stored.
5. Output: You see your answer.

Input & Output

- Input: Math expressions, menu choices, queries for the AI, and image files.
- Output: Calculated answers, converted measurements, Graphs, and AI responses.

4. Implementation

Language: C (C99 Standard)

Compiler/IDE: GCC / CLion / VS Code / Dev-C++

External Dependencies: windows.h, comdlg32, libcurl / System Calls

Key Features

- Parser-Based Calculator: This Calculator can easily Handle equations with brackets () and decimal points and calculate the result by following Mathematical rules.
- Graphing Mode: Plots x vs y coordinates dynamically right in the console.

- AI agent: Sends images and text to Google's Gemini 2.5 Flash model and fetches answers to your queries.
- Converter Mode: Provides you with a list of conversions to choose from and then converts your input to your desired unit.

Code Snippet (Sending request to AI Agent)

```
// command to send JSON data to Google Gemini API
```

```
sprintf(command,
```

```
"curl -s -X POST
```

```
"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-  
flash:generateContent?key=%s\" "
```

```
"-H \"Content-Type: application/json\" "
```

```
"-d @request.json > response.txt",
```

```
API_KEY);
```

```
// Running the command via system call
```

```
int status = system(command);
```

```
if (status == 0) {
```

```
    // Open response.html and parse the JSON result
```

```
// ...
```

```
}
```

Sample Output (Graphing Mode)

```
=====
==
```

FAST CALCULATOR PROJECT - GRAPHING MODE

```
=====
==
```

Enter function of x: $x*x$

Plotting: $y = x*x$

```
*   |   *
*   |   *
*   |   *
**  |  **
*****
```

5. Testing & Results

We tested everything against different scenarios to make sure it's stable, especially when it comes to the order of mathematical operations and connecting to the internet.

We tested all the modes repeatedly using different scenarios. Everything worked fine and we refined the code by debugging small errors/bugs.

6. Conclusion, Limitations & References

Conclusion

This project not only demonstrates the practical use of programming techniques but also to think and develop solutions for the betterment of

people. This project made it quite easy to study Calculus for students by assembling all different types of calculators into one single entity.

Limitations

- Internet access is required to use the Ai agent.
- Since we're drawing graphs with text characters (ASCII), the graph quality is low.
- Since we used windows.h for the console output customization, so the code won't work on Linux or MacOS without some changes.

Future Enhancements

- Add a Real GUI.
- Save History of each user's calculations.
- Improve graph quality through real UI and providing support for trigonometric functions in the graphing mode.

References

- Documentation: Google Gemini API Docs (AI Studio).
- Algorithm: Edsger Dijkstra's Shunting-yard Algorithm (Wikipedia).
- Book: The C Programming Language by Brian Kernighan and Dennis Ritchie.
- Resources: StackOverflow community for help with curl command syntax.